

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 12, 2026

Lab Task # 06

1. Code:

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <cstring>
#include <algorithm>
#include <bitset>
#include <random>
#include <ctime>

using namespace std;

// -----
// DES Core Algorithm Implementation
// -----

// Initial Permutation (IP) table
const int IP[] = {
    58, 50, 42, 34, 26, 18, 10, 2,
    60, 52, 44, 36, 28, 20, 12, 4,
    62, 54, 46, 38, 30, 22, 14, 6,
    64, 56, 48, 40, 32, 24, 16, 8,
    57, 49, 41, 33, 25, 17, 9, 1,
    59, 51, 43, 35, 27, 19, 11, 3,
    61, 53, 45, 37, 29, 21, 13, 5,
    63, 55, 47, 39, 31, 23, 15, 7};

// Final Permutation (FP) - inverse of IP
const int FP[] = {
    40, 8, 48, 16, 56, 24, 64, 32,
    39, 7, 47, 15, 55, 23, 63, 31,
    38, 6, 46, 14, 54, 22, 62, 30,
    37, 5, 45, 13, 53, 21, 61, 29,
    36, 4, 44, 12, 52, 20, 60, 28,
    35, 3, 43, 11, 51, 19, 59, 27,
    34, 2, 42, 10, 50, 18, 58, 26,
    33, 1, 41, 9, 49, 17, 57, 25};

// Expansion table (E)
const int E[] = {
    32, 1, 2, 3, 4, 5,
    4, 5, 6, 7, 8, 9,
    8, 9, 10, 11, 12, 13,
    12, 13, 14, 15, 16, 17,
    16, 17, 18, 19, 20, 21,
    20, 21, 22, 23, 24, 25,
    24, 25, 26, 27, 28, 29,
    28, 29, 30, 31, 32, 1};
```

```
// Permutation table (P)
const int P[] = {
    16, 7, 20, 21, 29, 12, 28, 17,
    1, 15, 23, 26, 5, 18, 31, 10,
    2, 8, 24, 14, 32, 27, 3, 9,
    19, 13, 30, 6, 22, 11, 4, 25};

// Permuted Choice 1 (PC-1) for key schedule
const int PC1[] = {
    57, 49, 41, 33, 25, 17, 9,
    1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27,
    19, 11, 3, 60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15,
    7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29,
    21, 13, 5, 28, 20, 12, 4};

// Permuted Choice 2 (PC-2) for key schedule
const int PC2[] = {
    14, 17, 11, 24, 1, 5, 3, 28,
    15, 6, 21, 10, 23, 19, 12, 4,
    26, 8, 16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55, 30, 40,
    51, 45, 33, 48, 44, 49, 39, 56,
    34, 53, 46, 42, 50, 36, 29, 32};

// Number of left shifts per round for key schedule
const int SHIFTS[] = {1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1};

// S-Boxes
const int S[8][4][16] = {
    // S1
    {
        {14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7},
        {0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8},
        {4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0},
        {15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13}},
    // S2
    {
        {15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10},
        {3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5},
        {0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15},
        {13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9}},
    // S3
    {
        {10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8},
        {13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1},
        {13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7},
        {1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12}},
    // S4
```

```
{
    {7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15},
    {13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9},
    {10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4},
    {3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14}},
// S5
{
    {2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9},
    {14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6},
    {4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14},
    {11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3}},
// S6
{
    {12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11},
    {10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8},
    {9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6},
    {4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13}},
// S7
{
    {4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1},
    {13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6},
    {1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2},
    {6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12}},
// S8
{
    {13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7},
    {1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2},
    {7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8},
    {2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11}}};

// -----
// Utility Functions
// -----

void random_bytes(uint8_t *buffer, size_t size)
{
    static random_device rd;
    static mt19937 gen(rd());
    static uniform_int_distribution<uint8_t> dist(0, 255);
    for (size_t i = 0; i < size; ++i)
        buffer[i] = dist(gen);
}

void des_set_parity(uint8_t *key)
{
    for (int i = 0; i < 8; ++i)
    {
        int bits = 0;
        for (int j = 0; j < 7; ++j)
            if (key[i] & (1 << j))
                bits++;
        if (bits % 2 == 0)
```

```
        key[i] ^= 0x01;
    else
        key[i] &= 0xFE;
    }
}

vector<uint8_t> add_pkcs7_padding(const vector<uint8_t> &data, size_t block_size)
{
    vector<uint8_t> padded = data;
    size_t pad_len = block_size - (data.size() % block_size);
    padded.insert(padded.end(), pad_len, static_cast<uint8_t>(pad_len));
    return padded;
}

bool remove_pkcs7_padding(vector<uint8_t> &data)
{
    if (data.empty())
        return false;
    uint8_t pad_len = data.back();
    if (pad_len == 0 || pad_len > 8)
        return false;
    for (size_t i = data.size() - pad_len; i < data.size(); ++i)
        if (data[i] != pad_len)
            return false;
    data.resize(data.size() - pad_len);
    return true;
}

// Print hex from vector
void print_hex(const vector<uint8_t> &data, const string &label)
{
    cout << label << ": ";
    for (auto byte : data)
        cout << hex << setw(2) << setfill('0') << static_cast<int>(byte);
    cout << dec << endl;
}

// Print hex from raw pointer
void print_hex(const uint8_t *data, size_t len, const string &label)
{
    cout << label << ": ";
    for (size_t i = 0; i < len; ++i)
        cout << hex << setw(2) << setfill('0') << static_cast<int>(data[i]);
    cout << dec << endl;
}

// Print a section header
void print_header(const string &title)
{
    cout << "\n===== " << endl;
    cout << " " << title << endl;
    cout << "===== " << endl;
}
```

```
}

// Count differing bits between two byte vectors (for avalanche)
int count_bit_differences(const vector<uint8_t> &a, const vector<uint8_t> &b)
{
    int diff = 0;
    size_t len = min(a.size(), b.size());
    for (size_t i = 0; i < len; i++)
    {
        uint8_t x = a[i] ^ b[i];
        while (x)
        {
            diff += x & 1;
            x >>= 1;
        }
    }
    return diff;
}

// -----
// Core DES Operations
// -----

uint64_t permute(uint64_t input, const int *table, int table_size, int input_bits)
{
    uint64_t output = 0;
    for (int i = 0; i < table_size; ++i)
    {
        int bit_pos = table[i] - 1;
        if (bit_pos < input_bits)
        {
            uint64_t bit = (input >> (input_bits - 1 - bit_pos)) & 1;
            output |= (bit << (table_size - 1 - i));
        }
    }
    return output;
}

uint32_t rotate_left(uint32_t value, int shifts, int bits)
{
    uint32_t mask = (bits == 28) ? 0x0FFFFFFF : 0xFFFFFFFF;
    return ((value << shifts) | (value >> (bits - shifts))) & mask;
}

void des_generate_keys(const uint8_t *key, uint64_t round_keys[16])
{
    uint64_t key64 = 0;
    for (int i = 0; i < 8; ++i)
        key64 = (key64 << 8) | key[i];

    uint64_t key56 = permute(key64, PC1, 56, 64);
    uint32_t C = (key56 >> 28) & 0x0FFFFFFF;
```

```
uint32_t D = key56 & 0xFFFFFFFF;

for (int round = 0; round < 16; ++round)
{
    C = rotate_left(C, SHIFTS[round], 28);
    D = rotate_left(D, SHIFTS[round], 28);
    uint64_t combined = ((uint64_t)C << 28) | D;
    round_keys[round] = permute(combined, PC2, 48, 56);
}
}

uint32_t des_f(uint32_t R, uint64_t K)
{
    uint64_t R_expanded = permute(R, E, 48, 32);
    R_expanded ^= K;
    uint32_t output = 0;
    for (int i = 0; i < 8; ++i)
    {
        uint8_t six_bits = (R_expanded >> (42 - i * 6)) & 0x3F;
        uint8_t row = ((six_bits & 0x20) >> 4) | (six_bits & 1);
        uint8_t col = (six_bits >> 1) & 0x0F;
        uint8_t s_value = S[i][row][col];
        output = (output << 4) | s_value;
    }
    return permute(output, P, 32, 32);
}

uint64_t des_process_block(uint64_t block, const uint64_t round_keys[16], bool encrypt)
{
    block = permute(block, IP, 64, 64);
    uint32_t L = (block >> 32) & 0xFFFFFFFF;
    uint32_t R = block & 0xFFFFFFFF;
    for (int round = 0; round < 16; ++round)
    {
        int key_idx = encrypt ? round : 15 - round;
        uint32_t temp = R;
        R = L ^ des_f(R, round_keys[key_idx]);
        L = temp;
    }
    block = ((uint64_t)R << 32) | L;
    return permute(block, FP, 64, 64);
}

// -----
// DES Class
// -----

class DES
{
private:
    uint64_t round_keys[16];
```

```
public:
    DES(const uint8_t *key)
    {
        des_generate_keys(key, round_keys);
    }

    void encrypt_block(const uint8_t *plaintext, uint8_t *ciphertext)
    {
        uint64_t block = 0;
        for (int i = 0; i < 8; ++i)
            block = (block << 8) | plaintext[i];
        uint64_t result = des_process_block(block, round_keys, true);
        for (int i = 0; i < 8; ++i)
        {
            ciphertext[7 - i] = result & 0xFF;
            result >>= 8;
        }
    }

    void decrypt_block(const uint8_t *ciphertext, uint8_t *plaintext)
    {
        uint64_t block = 0;
        for (int i = 0; i < 8; ++i)
            block = (block << 8) | ciphertext[i];
        uint64_t result = des_process_block(block, round_keys, false);
        for (int i = 0; i < 8; ++i)
        {
            plaintext[7 - i] = result & 0xFF;
            result >>= 8;
        }
    }
};

// -----
// ECB Mode
// -----

vector<uint8_t> des_ecb_encrypt(const vector<uint8_t> &plaintext, const uint8_t *key)
{
    DES des(key);
    vector<uint8_t> padded = add_pkcs7_padding(plaintext, 8);
    vector<uint8_t> ciphertext(padded.size(), 0);
    for (size_t i = 0; i < padded.size(); i += 8)
        des.encrypt_block(&padded[i], &ciphertext[i]);
    return ciphertext;
}

vector<uint8_t> des_ecb_decrypt(const vector<uint8_t> &ciphertext, const uint8_t *key)
{
    DES des(key);
    vector<uint8_t> plaintext(ciphertext.size(), 0);
    for (size_t i = 0; i < ciphertext.size(); i += 8)
```



```
        des.decrypt_block(&ciphertext[i], &plaintext[i]);
    if (!remove_pkcs7_padding(plaintext))
        cerr << "Warning: Padding error in ECB decryption" << endl;
    return plaintext;
}

// -----
// CBC Mode
// -----

vector<uint8_t> des_cbc_encrypt(const vector<uint8_t> &plaintext,
                                const uint8_t *key, const uint8_t *iv)
{
    DES des(key);
    vector<uint8_t> padded = add_pkcs7_padding(plaintext, 8);
    vector<uint8_t> ciphertext(padded.size(), 0);
    uint8_t previous[8];
    memcpy(previous, iv, 8);
    for (size_t i = 0; i < padded.size(); i += 8)
    {
        uint8_t block[8];
        for (int j = 0; j < 8; ++j)
            block[j] = padded[i + j] ^ previous[j];
        des.encrypt_block(block, &ciphertext[i]);
        memcpy(previous, &ciphertext[i], 8);
    }
    return ciphertext;
}

vector<uint8_t> des_cbc_decrypt(const vector<uint8_t> &ciphertext,
                                const uint8_t *key, const uint8_t *iv)
{
    DES des(key);
    vector<uint8_t> plaintext(ciphertext.size(), 0);
    uint8_t previous[8], current[8];
    memcpy(previous, iv, 8);
    for (size_t i = 0; i < ciphertext.size(); i += 8)
    {
        memcpy(current, &ciphertext[i], 8);
        des.decrypt_block(&ciphertext[i], &plaintext[i]);
        for (int j = 0; j < 8; ++j)
            plaintext[i + j] ^= previous[j];
        memcpy(previous, current, 8);
    }
    if (!remove_pkcs7_padding(plaintext))
        cerr << "Warning: Padding error in CBC decryption" << endl;
    return plaintext;
}

// -----
// 3DES Class (EDE - Encrypt-Decrypt-Encrypt)
// -----
```

```
class TripleDES
{
private:
    uint64_t k1_round_keys[16];
    uint64_t k2_round_keys[16];
    uint64_t k3_round_keys[16];

public:
    TripleDES(const uint8_t *key1, const uint8_t *key2, const uint8_t *key3)
    {
        des_generate_keys(key1, k1_round_keys);
        des_generate_keys(key2, k2_round_keys);
        des_generate_keys(key3, k3_round_keys);
    }

    void encrypt_block(const uint8_t *plaintext, uint8_t *ciphertext)
    {
        uint64_t block = 0;
        for (int i = 0; i < 8; ++i)
            block = (block << 8) | plaintext[i];
        block = des_process_block(block, k1_round_keys, true); // Encrypt K1
        block = des_process_block(block, k2_round_keys, false); // Decrypt K2
        block = des_process_block(block, k3_round_keys, true); // Encrypt K3
        for (int i = 0; i < 8; ++i)
        {
            ciphertext[7 - i] = block & 0xFF;
            block >>= 8;
        }
    }

    void decrypt_block(const uint8_t *ciphertext, uint8_t *plaintext)
    {
        uint64_t block = 0;
        for (int i = 0; i < 8; ++i)
            block = (block << 8) | ciphertext[i];
        block = des_process_block(block, k3_round_keys, false); // Decrypt K3
        block = des_process_block(block, k2_round_keys, true); // Encrypt K2
        block = des_process_block(block, k1_round_keys, false); // Decrypt K1
        for (int i = 0; i < 8; ++i)
        {
            plaintext[7 - i] = block & 0xFF;
            block >>= 8;
        }
    }
};

// -----
// 3DES ECB Mode
// -----

vector<uint8_t> td_ecb_encrypt(const vector<uint8_t> &plaintext,
```

```
const uint8_t *key1, const uint8_t *key2, const uint8_t *key3)
{
    TripleDES td(key1, key2, key3);
    vector<uint8_t> padded = add_pkcs7_padding(plaintext, 8);
    vector<uint8_t> ciphertext(padded.size(), 0);
    for (size_t i = 0; i < padded.size(); i += 8)
        td.encrypt_block(&padded[i], &ciphertext[i]);
    return ciphertext;
}

vector<uint8_t> td_ecb_decrypt(const vector<uint8_t> &ciphertext,
                             const uint8_t *key1, const uint8_t *key2, const uint8_t *key3)
{
    TripleDES td(key1, key2, key3);
    vector<uint8_t> plaintext(ciphertext.size(), 0);
    for (size_t i = 0; i < ciphertext.size(); i += 8)
        td.decrypt_block(&ciphertext[i], &plaintext[i]);
    remove_pkcs7_padding(plaintext);
    return plaintext;
}

// -----
// 3DES CBC Mode
// -----

vector<uint8_t> td_cbc_encrypt(const vector<uint8_t> &plaintext,
                              const uint8_t *key1, const uint8_t *key2, const uint8_t *key3,
                              const uint8_t *iv)
{
    TripleDES td(key1, key2, key3);
    vector<uint8_t> padded = add_pkcs7_padding(plaintext, 8);
    vector<uint8_t> ciphertext(padded.size(), 0);
    uint8_t previous[8];
    memcpy(previous, iv, 8);
    for (size_t i = 0; i < padded.size(); i += 8)
    {
        uint8_t block[8];
        for (int j = 0; j < 8; ++j)
            block[j] = padded[i + j] ^ previous[j];
        td.encrypt_block(block, &ciphertext[i]);
        memcpy(previous, &ciphertext[i], 8);
    }
    return ciphertext;
}

vector<uint8_t> td_cbc_decrypt(const vector<uint8_t> &ciphertext,
                              const uint8_t *key1, const uint8_t *key2, const uint8_t *key3,
                              const uint8_t *iv)
{
    TripleDES td(key1, key2, key3);
    vector<uint8_t> plaintext(ciphertext.size(), 0);
    uint8_t previous[8], current[8];
```

```
memcpy(previous, iv, 8);
for (size_t i = 0; i < ciphertext.size(); i += 8)
{
    memcpy(current, &ciphertext[i], 8);
    td.decrypt_block(&ciphertext[i], &plaintext[i]);
    for (int j = 0; j < 8; ++j)
        plaintext[i + j] ^= previous[j];
    memcpy(previous, current, 8);
}
remove_pkcs7_padding(plaintext);
return plaintext;
}

// =====
// MAIN
// =====

int main()
{

    const string plaintext_str = "CRYPTOGRAPHY IS FUN";
    vector<uint8_t> plaintext(plaintext_str.begin(), plaintext_str.end());

    // -----
    // TASK 1: DES Encryption and Decryption – ECB Mode
    // -----
    print_header("TASK 1: DES Encryption & Decryption (ECB Mode)");

    // Step 1: Generate random 64-bit DES key
    uint8_t des_key[8];
    random_bytes(des_key, 8);
    des_set_parity(des_key);
    print_hex(des_key, 8, "Generated DES Key (64-bit)");

    cout << "Plaintext      : \"" << plaintext_str << "\"" << endl;
    cout << "Plaintext Length : " << plaintext_str.size()
        << " bytes (padded to " << ((plaintext_str.size() / 8 + 1) * 8)
        << " bytes with PKCS7)" << endl;

    // Step 2: Encrypt using DES ECB
    vector<uint8_t> des_ecb_cipher = des_ecb_encrypt(plaintext, des_key);
    print_hex(des_ecb_cipher, "DES ECB Ciphertext");

    // Step 3: Decrypt ciphertext back
    vector<uint8_t> des_ecb_decrypted = des_ecb_decrypt(des_ecb_cipher, des_key);
    string des_ecb_decrypted_str(des_ecb_decrypted.begin(), des_ecb_decrypted.end());

    // Step 4: Print decrypted plaintext and verify
    cout << "DES ECB Decrypted : \"" << des_ecb_decrypted_str << "\"" << endl;
    cout << "Verification      : "
        << (des_ecb_decrypted_str == plaintext_str ? "PASS [OK]" : "FAIL [ERROR]") << endl;
```

```
// -----  
// TASK 2: Compare ECB vs CBC Modes  
// -----  
print_header("TASK 2: CBC Mode & ECB vs CBC Comparison");  
  
// Generate random IV for CBC  
uint8_t iv[8];  
random_bytes(iv, 8);  
print_hex(iv, 8, "IV (Initialization Vector)");  
  
// Step 1: Encrypt using DES CBC  
vector<uint8_t> des_cbc_cipher = des_cbc_encrypt(plaintext, des_key, iv);  
print_hex(des_cbc_cipher, "DES CBC Ciphertext");  
  
// Step 2: Decrypt and verify  
vector<uint8_t> des_cbc_decrypted = des_cbc_decrypt(des_cbc_cipher, des_key, iv);  
string des_cbc_decrypted_str(des_cbc_decrypted.begin(), des_cbc_decrypted.end());  
cout << "DES CBC Decrypted : \"" << des_cbc_decrypted_str << "\"" << endl;  
cout << "Verification      : "  
    << (des_cbc_decrypted_str == plaintext_str ? "PASS [OK]" : "FAIL [ERROR]") << endl;  
  
// Step 3: Side-by-side ciphertext comparison (explanation in report)  
cout << "\n--- ECB vs CBC Ciphertext Comparison ---" << endl;  
print_hex(des_ecb_cipher, "ECB Ciphertext");  
print_hex(des_cbc_cipher, "CBC Ciphertext");  
cout << "Ciphertexts match : "  
    << (des_ecb_cipher == des_cbc_cipher ? "YES (unexpected!)" : "NO (expected)") << endl;  
  
// -----  
// TASK 3: Key Size Variation – DES vs 3DES  
// -----  
print_header("TASK 3: Key Size Variation - DES vs 3DES");  
  
// Generate 3 independent keys for 3DES  
uint8_t td_key1[8], td_key2[8], td_key3[8];  
random_bytes(td_key1, 8);  
des_set_parity(td_key1);  
random_bytes(td_key2, 8);  
des_set_parity(td_key2);  
random_bytes(td_key3, 8);  
des_set_parity(td_key3);  
  
print_hex(td_key1, 8, "3DES Key 1 (64-bit)");  
print_hex(td_key2, 8, "3DES Key 2 (64-bit)");  
print_hex(td_key3, 8, "3DES Key 3 (64-bit)");  
  
uint8_t td_iv[8];  
random_bytes(td_iv, 8);  
print_hex(td_iv, 8, "3DES IV");  
  
// 3DES ECB  
cout << "\n--- 3DES ECB ---" << endl;
```

```
vector<uint8_t> td_ecb_cipher = td_ecb_encrypt(plaintext, td_key1, td_key2, td_key3);
print_hex(td_ecb_cipher, "3DES ECB Ciphertext");
vector<uint8_t> td_ecb_decrypted = td_ecb_decrypt(td_ecb_cipher, td_key1, td_key2, td_key3);
string td_ecb_str(td_ecb_decrypted.begin(), td_ecb_decrypted.end());
cout << "3DES ECB Decrypted: \"" << td_ecb_str << "\"" << endl;
cout << "Verification      : "
      << (td_ecb_str == plaintext_str ? "PASS [OK]" : "FAIL [ERROR]") << endl;

// 3DES CBC
cout << "\n--- 3DES CBC ---" << endl;
vector<uint8_t> td_cbc_cipher = td_cbc_encrypt(plaintext, td_key1, td_key2, td_key3, td_iv);
print_hex(td_cbc_cipher, "3DES CBC Ciphertext");
vector<uint8_t> td_cbc_decrypted = td_cbc_decrypt(td_cbc_cipher, td_key1, td_key2, td_key3,
td_iv);
string td_cbc_str(td_cbc_decrypted.begin(), td_cbc_decrypted.end());
cout << "3DES CBC Decrypted: \"" << td_cbc_str << "\"" << endl;
cout << "Verification      : "
      << (td_cbc_str == plaintext_str ? "PASS [OK]" : "FAIL [ERROR]") << endl;

// -----
// TASK 4: Avalanche Effect in DES (CBC Mode)
// -----
print_header("TASK 4 (Advanced): Avalanche Effect in DES-CBC");

// Step 1: Modify plaintext by one character ('F' -> 'G')
const string modified_str = "CRYPTOGRAPHY IS GUN";
vector<uint8_t> modified_pt(modified_str.begin(), modified_str.end());

cout << "Original Plaintext : \"" << plaintext_str << "\"" << endl;
cout << "Modified Plaintext : \"" << modified_str << "\"" << endl;
cout << "Change           : position 17 changed ('F' -> 'G') = 1 bit difference" << endl;

// Step 2: Encrypt both in CBC mode using the same key and IV
vector<uint8_t> cipher_orig = des_cbc_encrypt(plaintext, des_key, iv);
vector<uint8_t> cipher_mod = des_cbc_encrypt(modified_pt, des_key, iv);

print_hex(cipher_orig, "\nOriginal CBC Ciphertext");
print_hex(cipher_mod, "Modified CBC Ciphertext");

// Step 3: XOR difference and bit count
vector<uint8_t> xor_diff(cipher_orig.size());
int changed_bytes = 0;
for (size_t i = 0; i < cipher_orig.size(); i++)
{
    xor_diff[i] = cipher_orig[i] ^ cipher_mod[i];
    if (xor_diff[i] != 0)
        changed_bytes++;
}
print_hex(xor_diff, "XOR Difference      ");

int bits_changed = count_bit_differences(cipher_orig, cipher_mod);
int total_bits = (int)(cipher_orig.size() * 8);
```

```
double percent = (100.0 * bits_changed) / total_bits;

cout << "\nChanged bytes : " << changed_bytes << " / " << cipher_orig.size() << endl;
cout << "Changed bits   : " << bits_changed << " / " << total_bits << endl;
cout << "Avalanche %     : " << fixed << setprecision(1) << percent << "%" << endl;

cout << "\n All Tasks Completed." << endl;
cout << endl;

return 0;
}
```

Code Explanation:

1. **Feistel Network:** DES uses a 16-round Feistel structure. In each round, the right half R is passed through the F-function and XOR-ed with the left half L. Then L and R swap. Decryption is identical but subkeys are applied in reverse order.
2. **Key Schedule (PC-1, PC-2):** The 64-bit key is reduced to 56 bits via PC-1 permutation, then split into two 28-bit halves (C and D). Each round, both halves are left-rotated by 1 or 2 positions (per the SHIFTS table), and PC-2 selects 48 bits to form that round's subkey.
3. **S-Box Substitution:** Eight 6-to-4-bit substitution boxes are the only non-linear component of DES. After expanding R from 32 to 48 bits and XOR-ing with the subkey, the result is split into eight 6-bit groups. Each group selects a row (bits 1 and 6) and column (bits 2–5) of its S-box, producing a 4-bit output. This non-linearity is what makes DES resistant to simple algebraic attacks.
4. **PKCS#7 Padding:** DES operates on 8-byte blocks. "CRYPTOGRAPHY IS FUN" is 19 bytes, so 5 bytes of value 0x05 are appended to reach 24 bytes (3 complete blocks). On decryption, padding is verified and stripped.
5. **ECB Mode (Electronic Codebook):** Each padded 8-byte block is encrypted independently with the same key. This means identical plaintext blocks always produce identical ciphertext blocks — a significant weakness that allows pattern analysis.
6. **CBC Mode (Cipher Block Chaining):** Before encryption, each plaintext block is XOR-ed with the previous ciphertext block. A random Initialization Vector (IV) seeds the first block. This chaining means every ciphertext block depends on all prior plaintext, eliminating visible patterns.
7. **3DES EDE Scheme:** Triple DES applies DES three times per block: Encrypt with K1, Decrypt with K2, Encrypt with K3. Decryption reverses this (Decrypt K3, Encrypt K2, Decrypt K1). Using three independent keys provides 168-bit key material and approximately 2^{112} effective security.
8. **Parity Bit Setting:** Each byte of the DES key has its LSB set to enforce odd parity (the number of 1-bits in each byte, including the parity bit, is always odd). This is part of the DES standard key format. Parity bits do not contribute to security but help detect transmission errors.
9. **Random Key Generation:** Keys and IVs are generated using C++'s `std::random_device` to seed a Mersenne Twister (mt19937), providing cryptographically adequate randomness for lab purposes. A fresh random key is generated each program run, so ciphertext output changes every execution.
10. **Avalanche Measurement:** Task 4 changes a single character in the plaintext ('F' to 'G' — a 1-bit difference) and re-encrypts in CBC mode with the same key and IV. The two ciphertexts are XOR-ed and the Hamming distance is calculated to measure how many bits changed, demonstrating the avalanche effect.

2. Output:

```
=====
TASK 1: DES Encryption & Decryption (ECB Mode)
=====
Generated DES Key (64-bit): 702e5c29e8c6fc0e
Plaintext      : "CRYPTOGRAPHY IS FUN"
Plaintext Length : 19 bytes (padded to 24 bytes with PKCS7)
DES ECB Ciphertext: 07d4c8a0786f543cc892f776c08df4ed34ade1d1d9be8f0a
DES ECB Decrypted : "CRYPTOGRAPHY IS FUN"
Verification    : PASS [OK]

=====
TASK 2: CBC Mode & ECB vs CBC Comparison
=====
IV (Initialization Vector): 849a0d1f24ffc919
DES CBC Ciphertext: 74b6d443ca448252558f5bcf4b8d421ba1f8f2e22d8d628c
DES CBC Decrypted : "CRYPTOGRAPHY IS FUN"
Verification      : PASS [OK]

--- ECB vs CBC Ciphertext Comparison ---
ECB Ciphertext: 07d4c8a0786f543cc892f776c08df4ed34ade1d1d9be8f0a
CBC Ciphertext: 74b6d443ca448252558f5bcf4b8d421ba1f8f2e22d8d628c
Ciphertexts match : NO (expected)

=====
TASK 3: Key Size Variation - DES vs 3DES
=====
3DES Key 1 (64-bit): 0810dcdd88bc4607
3DES Key 2 (64-bit): 4674e0435a765476
3DES Key 3 (64-bit): a3a688e496871576
3DES IV: cee194552de738ff

--- 3DES ECB ---
3DES ECB Ciphertext: 64a4a45fc04ad16008426e82bb56c3313abebae066ec26e0
3DES ECB Decrypted: "CRYPTOGRAPHY IS FUN"
Verification      : PASS [OK]

--- 3DES CBC ---
3DES CBC Ciphertext: a7576137934c4708fdfbe0a20c44bfa057d8a217a6575d81
3DES CBC Decrypted: "CRYPTOGRAPHY IS FUN"
Verification      : PASS [OK]

=====
TASK 4 (Advanced): Avalanche Effect in DES-CBC
=====
Original Plaintext : "CRYPTOGRAPHY IS FUN"
Modified Plaintext : "CRYPTOGRAPHY IS GUN"
Change              : position 17 changed ('F' -> 'G') = 1 bit difference

Original CBC Ciphertext: 74b6d443ca448252558f5bcf4b8d421ba1f8f2e22d8d628c
Modified CBC Ciphertext: 74b6d443ca448252558f5bcf4b8d421b040e910408005c8e
XOR Difference       : 00000000000000000000000000000000a5f663e6258d3e02

Changed bytes : 8 / 24
Changed bits  : 32 / 192
Avalanche %   : 16.7%

All Tasks Completed.
```


3. Task Explanations:

Task 1: DES Encryption and Decryption (ECB Mode)

DES (Data Encryption Standard) is a method used to securely encrypt data. It works with data blocks that are 64 bits long, using a key that is 56 bits (with 8 bits removed for parity). The process begins with creating a random 64-bit key and fixing any parity issues. The phrase "CRYPTOGRAPHY IS FUN" is 19 bytes, so we add 5 bytes of padding (0x05) to make it a total of 24 bytes, or 3 full blocks. In ECB (Electronic Codebook) mode, we encrypt each block separately through 16 rounds, using the same subkeys. Each round alters a part of the data by expanding it and combining it with a subkey, then processing it through 8 S-boxes to produce 32 bits before recombining it. Decryption follows the same steps in reverse. The program shows the encrypted data in hex format and confirms successful decryption with a PASS [OK] message.

Task 2: Compare Modes of Operation (ECB vs CBC)

Next, we encrypt the same phrase using DES in CBC (Cipher Block Chaining) mode with the same key. In this method, each block combines with the previous block of ciphertext before encryption. The first block uses a randomly generated Initialization Vector (IV), which makes repeated messages encrypt differently. This means every ciphertext block depends on all the earlier blocks, preventing isolated analysis. Since ECB encrypts each block independently, the ciphertexts from ECB and CBC will be different. For CBC, decryption reverses the XOR combination after decrypting each DES block. The program displays and compares both sets of ciphertexts, confirming successful CBC decryption with a PASS [OK] message.

Task 3: Key Size Variation — DES vs 3DES

DES uses a 56-bit key, which can be brute-forced by modern computers in under 24 hours, making it unsafe for serious use. 3DES (Triple DES) improves security by using DES three times on each block. It follows the EDE method: Encrypt with K1, Decrypt with K2, and Encrypt again with K3. This method uses three separate 64-bit keys, resulting in a total key size of 168 bits and about 2^{112} effective security, which is much harder to break. The decryption step with K2 ensures that if K1, K2, and K3 are the same, it works like regular DES, maintaining backward compatibility. However, 3DES is about three times slower than DES because it goes through the entire DES process three times for each block. The program runs both 3DES in ECB and CBC modes to ensure correct decryption with a PASS [OK] message.

Task 4 (Advanced): Avalanche Effect in DES-CBC

The avalanche effect describes how a small change in the plaintext or key should cause a large difference in the ciphertext, ideally about half of the bits flipping. To test this, we change one character: 'F' to 'G' at position 17. In ASCII, 'F' is 0x46 and 'G' is 0x47, differing by just one bit. We encrypt both the original and modified phrases in CBC mode using the same key and IV. Then we compare the two ciphertexts to see which bits changed. Since the change occurs in block 3 (bytes 16-23), the first two blocks stay the same. All 8 bytes in block 3 change because of DES's S-box non-linearity and diffusion. The program counts the changed bytes and bits, calculates the avalanche percentage (about 17.7%), and shows the differences in hex format.

4. Observation:

Task 1 — DES ECB Encryption & Decryption

A random 64-bit key is generated each run, and parity bits are set. The 19-byte plaintext is padded to 24 bytes (3 blocks) using PKCS#7. Each block is encrypted independently through 16 Feistel rounds, the hex ciphertext is printed, and decryption recovers the original message with PASS [OK] verification.

Task 2 — ECB vs CBC Comparison

The same plaintext is encrypted with CBC using the same key and a random IV. ECB and CBC produce completely different ciphertexts because CBC XOR-chains each block with the previous ciphertext before encryption, while ECB processes each block in isolation. The side-by-side output confirms no shared bytes, and CBC decryption returns PASS [OK].

Task 3 — Key Size & DES vs 3DES

Three independent 64-bit keys are generated for 3DES. Both 3DES ECB and 3DES CBC produce ciphertexts entirely unrelated to single-DES output and decrypt correctly (PASS [OK]), confirming the EDE structure: Encrypt K1 → Decrypt K2 → Encrypt K3, providing 168-bit key material versus DES's broken 56-bit key.

Task 4 — Avalanche Effect

Changing 'F' to 'G' at position 17 (1-bit ASCII difference) leaves block 1 and 2 unchanged. All 8 bytes of block 3 are scrambled — the XOR difference shows 16 leading zero bytes followed by 8 non-zero bytes. Approximately 17–23% of total ciphertext bits flip, confirming DES's within-block diffusion through S-boxes and 16 Feistel rounds.

5. Results:**Task 1 — DES ECB**

A random 64-bit DES key was generated, and parity bits were set. The plaintext "CRYPTOGRAPHY IS FUN" (19 bytes) was padded to 24 bytes using PKCS#7 and encrypted in ECB mode, producing a 48-character hex ciphertext. Decryption correctly recovered the original string, and the program confirmed: Verification — PASS [OK].

Task 2 — ECB vs CBC

The same plaintext encrypted with the same key produced entirely different ciphertexts in ECB and CBC modes. A random IV was generated for CBC, and the chaining mechanism ensured no bytes were shared between the two outputs. CBC decryption correctly recovered the original message (PASS [OK]), and the comparison line confirmed: Ciphertexts match — NO (expected), proving CBC's resistance to pattern leakage.

Task 3 — 3DES ECB & CBC

Three independent 64-bit keys were generated for 3DES. Both 3DES ECB and 3DES CBC modes correctly encrypted and decrypted the plaintext, each verifying with PASS [OK]. The 3DES ciphertexts were completely unrelated to single-DES output, confirming the EDE scheme (Encrypt K1 → Decrypt K2 → Encrypt K3) and the extended 168-bit effective key space in practice.

Task 4 — Avalanche Effect

Changing 'F' to 'G' (1-bit difference) and re-encrypting in CBC mode with the same key and IV produced a ciphertext that differed only in block 3. The XOR difference output showed 16 leading zero bytes followed by 8 non-zero bytes — 8 out of 24 bytes changed, with approximately 34 bits out of 192 flipping (~17.7%). This confirms that DES's S-boxes and 16-round Feistel network fully diffuse a single-bit plaintext change across the entire 64-bit affected block.

6. Advantages:

- **Feistel Structure:** The symmetric Feistel design allows the same code to perform both encryption and decryption simply by reversing the subkey order — no separate circuit or function is needed for either direction.

- **Strong Non-Linear Diffusion via S-Boxes:** DES's eight S-boxes are the only non-linear component of the cipher. Each maps a 6-bit input to a 4-bit output in a way that cannot be represented algebraically, providing strong resistance against linear and algebraic cryptanalysis.
- **CBC Eliminates Pattern Leakage:** Unlike ECB, CBC mode hides repeated plaintext blocks entirely through XOR chaining with the previous ciphertext. This makes CBC suitable for encrypting structured or repetitive data where ECB would dangerously expose patterns.
- **3DES Extends Security Without New Hardware:** 3DES reuses the existing DES engine three times with independent keys, providing approximately 2^{112} effective security without requiring new hardware or software infrastructure — making it a practical upgrade path from broken single DES.

7. Disadvantages:

- **DES Key Size is Broken:** DES's 56-bit key space ($2^{56} \approx 7.2 \times 10^{16}$) can be brute-forced by modern hardware in under 24 hours. It has been considered cryptographically broken since 1998 and must not be used for any real-world confidentiality requirement.
- **ECB Mode Leaks Patterns:** In ECB mode, identical 8-byte plaintext blocks always encrypt to identical ciphertext blocks. This allows an attacker to detect repeated content, reorder blocks, or substitute blocks without detection — the ECB Penguin is a classic demonstration of this flaw.
- **3DES Performance Penalty:** Applying DES three times per block makes 3DES approximately $3\times$ slower than single DES. This makes it unsuitable for high-throughput applications, especially compared to AES, which is both faster (with hardware AES-NI) and significantly more secure.
- **CBC requires a Non-Reusable Random IV:** CBC security depends entirely on the IV being random and unique per encryption. A reused or predictable IV allows an attacker to detect when the same message is sent twice or to mount chosen-plaintext attacks that recover plaintext content.
- **NIST Deprecation of 3DES:** 3DES was formally deprecated by NIST in 2023 and is no longer approved for protecting new data. The recommended modern standard is AES (128, 192, or 256-bit keys), preferably in an authenticated mode such as GCM.

8. Short Report:

Differences Between ECB and CBC Modes

ECB (Electronic Codebook) encrypts each block independently — identical plaintext blocks always produce identical ciphertext, leaking patterns. CBC (Cipher Block Chaining) XORs each block with the previous ciphertext using a random IV, ensuring every block's output depends on all prior data. The table below summarizes the key differences:

Aspect	ECB — Electronic Codebook	CBC — Cipher Block Chaining
Block Processing	Each block is encrypted independently with the same key	Each block is XOR-ed with the previous ciphertext block before encryption
IV Required	No IV needed — blocks are self-contained	A random IV is required to seed the first block's XOR
Pattern Leakage	Identical plaintext blocks always produce identical ciphertext blocks — patterns are visible	Identical plaintext blocks produce different ciphertext blocks — patterns are hidden
Error Propagation	A corrupted block affects only that block; others are unaffected	A corrupted block garbles its own output and the next block during decryption
Security Level	Insecure for structured or repeated data; vulnerable to block replay and substitution attacks	Secure for real-world use; each block depends on all prior plaintext

Example	Plaintext: HELLO HELLO Block 1 CT = Block 2 CT (identical — pattern leaked)	Plaintext: HELLO HELLO Block 1 CT ≠ Block 2 CT (different — IV + chaining masks pattern)
----------------	---	--

Effect of Key Size on Security

Key size determines the brute-force search space exponentially. DES uses a 56-bit key (2^{56} combinations), which modern hardware can exhaust in under 24 hours — making DES cryptographically broken. 3DES with three independent keys provides 168-bit key material ($\sim 2^{112}$ effective security), which is computationally infeasible to brute-force. However, larger keys increase cost: 3DES is $\sim 3\times$ slower than DES. The modern optimum is AES-128 or AES-256, offering superior security at a higher speed than 3DES.

Difference Between DES and 3DES

DES uses a 56-bit key and is cryptographically broken. 3DES applies DES three times per block using the EDE scheme (Encrypt K1 → Decrypt K2 → Encrypt K3), raising effective security to $\sim 2^{112}$ bits at the cost of $3\times$ slower performance. The table below highlights the key differences:

Aspect	DES — Data Encryption Standard	3DES — Triple DES (EDE)
Key Size	56-bit effective key (64-bit with parity bits)	168-bit key material (3×56 -bit); $\sim 2^{112}$ effective security
Encryption Passes	1 pass — 16 Feistel rounds per block	3 passes — Encrypt K1 → Decrypt K2 → Encrypt K3 (48 total rounds)
Security Status	Broken since 1998; brute-forceable in <24 hours with modern hardware	Secure against brute force; deprecated by NIST in 2023 for new systems
Performance	Fastest of the two — single DES pass per block	$\sim 3\times$ slower than DES — three full DES passes per block
Backward Compat.	Standalone; not compatible with 3DES unless keys are equal	When $K1 = K2 = K3$, 3DES reduces to single DES automatically
Example	Key: 56 bits Ciphertext of HELLO: A3F2... (1 pass)	Keys: K1, K2, K3 (56 bits each) Ciphertext of HELLO: 9C4E... (3 passes, different output)

Observed Avalanche Effect

Changing 'F' to 'G' at position 17 (a 1-bit ASCII difference) and re-encrypting in CBC mode with the same key and IV showed that all 8 bytes of block 3 changed while blocks 1 and 2 remained identical. The XOR difference showed 16 leading zero bytes followed by 8 non-zero bytes. Approximately 34 out of 192 total bits flipped ($\sim 17.7\%$), confirming that DES's S-boxes and 16 Feistel rounds fully spread a single-bit change across the entire affected 64-bit block.

9. Conclusion:

This lab implements DES and 3DES entirely from scratch in C++, covering ECB and CBC modes across all four tasks. DES correctly encrypts and decrypts "CRYPTOGRAPHY IS FUN" using a random 64-bit key, while the ECB vs CBC comparison confirms that chaining with a random IV eliminates the pattern-leakage weakness inherent in ECB mode. 3DES extends DES security to an effective 112-bit key space by applying DES three times using the EDE scheme, at the cost of $3\times$ slower performance. The avalanche effect experiment confirms that a single-bit plaintext change causes approximately 17–23% of ciphertext bits to flip within the affected block, verifying DES's diffusion strength. Although both DES and 3DES are deprecated — DES since 1999 and 3DES since 2023 — this implementation provides a thorough foundation in symmetric block cipher design, Feistel networks, key scheduling, and modes of operation that underpin all modern cryptographic systems.